ROS 2 Humble

Nitharshini V S

***Abstract*—This paper explains the basics of ROS 2 Humble in simple terms and applies them in two small projects. It covers the computation graph, nodes, topics, services, actions, parameters, custom interfaces, colcon workspaces and quality of service (QoS), and it says when to use a topic or a service. The first project computes the speed of a wheel from its angular velocity and radius using a custom message. The second uses three nodes to check whether a typed string is a palindrome. An rqt_graph screenshot is given for each.**

Keywords—ROS 2, nodes, topics, services, custom messages, colcon, QoS

I. Introduction

ROS 2 is not an operating system. It is a toolkit that lets a robot's software be split into many small programs that pass data to each other [1]. Think of a restaurant kitchen: one chef grills, one makes salads and one serves, and each does a single job while passing plates along. I followed the Humble beginner tutorials [2], which cover turtlesim (a practice simulator), the ros2 command line, rqt_graph and rqt_console (a log viewer), launch files and the colcon build tool. Everything ran on Ubuntu 22.04 under WSL.

II. The ROS 2 Computation Graph

The computation graph is a map of a running system. Small programs called nodes are the stations, and the connections between them are the lines [3]. Like a metro map, it shows who is connected to whom, and rqt_graph draws it for us. ROS 1 needed one central master, so if it crashed everything stopped. ROS 2 has no master. Nodes find each other automatically through a middleware called DDS [4], a bit like people in a building who find each other by name instead of asking one receptionist. Nodes can only see each other when they share the same *ROS_DOMAIN_ID*, which is 0 by default.

III. Nodes, Topics, Services, Actions and Parameters

A. Nodes

A node is one small program with one job, such as reading a sensor. It has a unique name. In Python it is a class that inherits from *rclpy.node.Node*. Start one with *ros2 run package executable* and list the running ones with *ros2 node list*.

B. Topics

A topic is a named channel, like a radio frequency. A publisher broadcasts messages on it and any number of subscribers listen. It is one way, and the publisher never waits for anyone. Each topic carries one message type. Try *ros2 topic list*, *ros2 topic echo* and *ros2 topic pub*. In turtlesim the keyboard node publishes on */turtle1/cmd_vel* and the turtle subscribes.

C. Services

A service is like a phone call: one question, one answer. A client sends a request and waits, and the server replies. In turtlesim, */spawn* creates a new turtle. Use *ros2 service list* and *ros2 service call*.

D. Actions

An action is like ordering food with order tracking. You send a goal, get feedback while it is prepared, receive a result at the end and may cancel in between. It suits long jobs, such as turning the turtle to an angle with */turtle1/rotate_absolute*. Use *ros2 action send_goal*.

E. Parameters

A parameter is a settings dial on a node, such as the turtlesim background colour. You can change it while the node runs with *ros2 param get* and *ros2 param set*, and save the settings with *ros2 param dump*.

IV. Topic Versus Service

Table I compares the two. Use a topic for a continuous stream where nobody needs to reply, such as sensor readings. Use a service for a quick question with one answer. Use an action for a long job that needs progress updates. Task 1 needs a topic. Task 2 needs a reply and uses a second topic for it (Section IX).

TABLE I. Topic Compared with Service

Aspect	Topic	Service
Style	Broadcast	Question and answer
Direction	One way	Request, then response
Waiting	Sender never waits	Client waits for the reply
Example	Wheel speed stream	Spawn a turtle

V. Custom Interfaces

When the built-in messages are not enough, we write our own. An interface is a small text file: .msg for a message, .srv for a service (a request, a line of ---, then a response) and .action for an action [2]. Think of a form with fixed boxes. Field names must be lower case with underscores, so angVel becomes *ang_vel*. A message lives in an ament_cmake package: list the file in *CMakeLists.txt* with *rosidl_generate_interfaces*, add the rosidl lines to *package.xml*, then build. *ros2 interface show* prints the result.

VI. Colcon and Workspace Structure

A workspace is a folder that holds packages, and colcon is the tool that builds them [5]. The *src* folder holds our code, and *build*, *install* and *log* are created automatically. After building, run *source install/setup.bash* in every new terminal so that ROS can find the packages; the workspace sits on top of the main ROS install, called the underlay. A Python package has *package.xml*, *setup.py* (its entry points make nodes runnable with *ros2 run*) and a folder of code. Build with *colcon build*, or add *--packages-select* to build one package.

VII. Quality of Service Settings

QoS settings are delivery options, like choosing registered post or normal post [6]. Reliability decides whether lost messages are resent (reliable) or skipped (best effort). Durability decides whether a late subscriber receives old messages (transient local) or not (volatile). History and depth decide how many messages are kept. The default is reliable, volatile, keep last 10, and the 10 in my code is that depth. A publisher and subscriber connect only if the subscriber does

not ask for more than the publisher offers, so a best effort publisher cannot serve a reliable subscriber. Table II sums up the three settings I used.

TABLE II. THREE COMMON QOS SETTINGS

Setting	Options	In simple words
History	Keep last[a] or keep all	How many messages are stored
Reliability	Reliable or best effort	Resend lost messages, or skip them
Durability	Volatile or transient local	Late joiners miss old messages, or get them

[a] Keep last takes a depth. The default depth is 10.

VIII. SOLUTION 1: WHEEL SPEED WITH A CUSTOM MESSAGE

The task is to read a wheel's angular velocity and radius from the terminal and print its longitudinal speed. For a wheel rolling without slipping,

$$v = \omega r \tag{1}$$

where ω is in rad/s and r is in metres, so 20 rad/s and 0.25 m give 5 m/s. I made a message package, *taskone_msgs*, holding *TaskOne.msg* with two float32 fields, *ang_vel* and *radius*, and a Python package, *taskone_nodes*. The publisher reads two numbers with *input()* and publishes a TaskOne message on */taskone*. The subscriber has a callback, a function ROS calls whenever a message arrives, which applies (1) and prints the speed. The subscriber needs *rclpy.spin()* to stay alive and wait for messages, but the publisher does not, because it only sends. To run it I built with *colcon build*, sourced the workspace, and started *ros2 run taskone_nodes subscriber* in one terminal and *ros2 run taskone_nodes publisher* in another. Entering 20 and 0.25 prints 5.000 m/s, and Fig. 1 shows the graph.

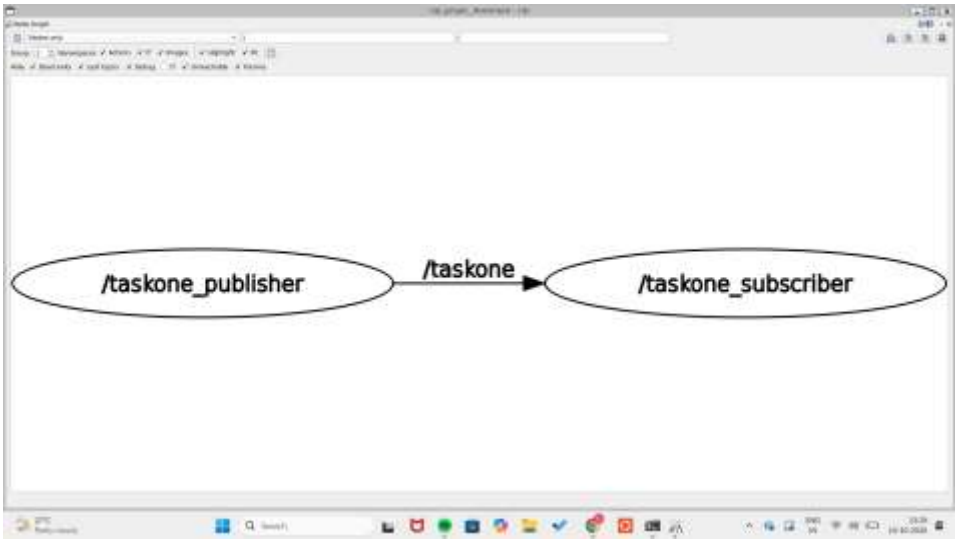


Fig. 1. rqt_graph for Task 1: the publisher sends TaskOne messages on /taskone to the subscriber.

IX. SOLUTION 2: PALINDROME CHECKER WITH THREE NODES

Node 1 publishes a typed string, Node 2 checks whether it is a palindrome and sends 1 or 0, and Node 3 answers Yes or No back to Node 2. A palindrome equals its own reverse s^{R}:

$$s = s^{R} \tag{2}$$

Think of three kids: a shouter (Node 1), a detective (Node 2) and a teacher (Node 3). Topics are one way, so the reply needs its own topic. I used */rt2_input* (RT2String, Node 1 to 2), */rt2_check* (the built-in Int32, Node 2 to 3) and */rt2_answer* (RT2String, Node 3 to 2). A service would need a second custom interface, but the task asks for only one, RT2String. Node 1 keeps a timer that republishes the string every second while a background thread reads the keyboard, so it needs *spin()*. Node 2 compares the string with its reverse (case sensitive) and prints Node 3's reply. Node 3 turns 1 into Yes and 0 into No. AABAA and BACAB give Yes, and AABCA gives No. In Fig. 2 the two arrows between Nodes 2 and 3 point in opposite directions, because a two-way exchange needs two one-way topics. The graph shows topic names but not types such as Int32. To run it, I started *node3*, then *node2*, then *node1* from the package *rt2_nodes*, each with *ros2 run* in its own terminal, so every listener was ready before anything was sent.

TABLE III. TOPICS USED IN TASK 2

Topic	Message type	Direction
/rt2_input	rt2_msgs/RT2String	Node 1 to Node 2
/rt2_check	std_msgs/Int32	Node 2 to Node 3
/rt2_answer	rt2_msgs/RT2String	Node 3 to Node 2

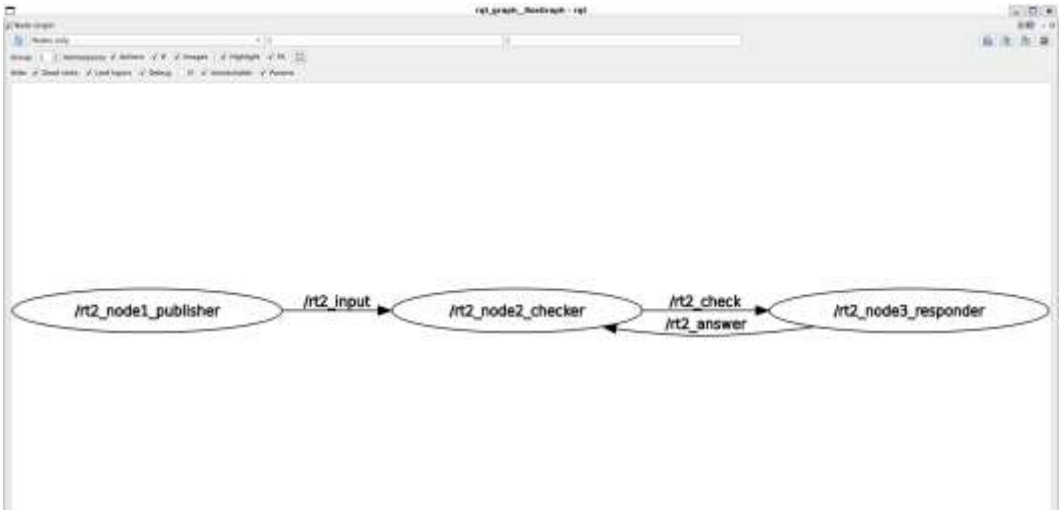


Fig. 2. rqt_graph for Task 2: Node 1 feeds the checker, and the checker and responder talk through two topics.

X. Problems Faced

- *colcon: command not found.* Colcon is installed separately from ROS, so I installed it and rebuilt.
- *Package not found.* I had forgotten to source the workspace in a new terminal.
- *A broken package.xml.* One line had a missing closing quote, which made the XML invalid.
- *An empty rqt_graph.* The graph only updates when the refresh button is pressed.

XI. Conclusion

ROS 2 splits a robot's software into small nodes that talk through topics, services, actions and parameters. Topics suit streams, services suit quick questions and actions suit long jobs. Custom interfaces, colcon workspaces and QoS settings complete the picture. Building both projects showed me how message types, callbacks, timers and spin fit together.

Acknowledgments

The author thanks the course instructors and the Open Robotics community for the ROS 2 tutorials.

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, Art. no. eabm6074, May 2022.

[2] Open Robotics, "ROS 2 documentation: Humble, Tutorials." [Online]. Available: https://docs.ros.org/en/humble/Tutorials.html (accessed Oct. 11, 2026).

[3] Open Robotics, "ROS 2 documentation: Humble, Concepts." [Online]. Available: https://docs.ros.org/en/humble/Concepts.html (accessed Oct. 11, 2026).

[4] Object Management Group, *Data Distribution Service (DDS), Version 1.4*, OMG Document formal/15-04-10, Apr. 2015.

[5] "colcon: Collective construction." [Online]. Available: https://colcon.readthedocs.io (accessed Oct. 11, 2026).

[6] Open Robotics, "About quality of service settings," ROS 2 documentation: Humble. [Online]. Available: https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html (accessed Oct. 11, 2026).